UNIVERSITAS CAKRAWALA

LAPORAN INDIVIDU

UJIAN TENGAH SEMESTER (UTS)

Hash Table & Linked List

Cookbook System - Aplikasi Manajemen Resep Berbasis Python

Nama	Ikmal Luthfi
NIM	24110300023
Mata Kuliah	Programming Fundamental
Program Studi	Ilmu Komputer
Dosen Pengampu	Fikri Hanif, S.T., M.S.M.
Semester / Kelas	Ganjil / Prog4
Tahun Akademik	2026/2027

Oktober 2026

1. Kontribusi dan Analisis IPO

Saya mengerjakan jalur pencarian berbasis ID di berkas data.py, terdiri dari hash_function() yang menghitung posisi resep, make_hash_table() yang membangun tabel dari data awal, dan get_recipe() yang mencari satu resep berdasarkan ID-nya. Struktur yang saya bangun ini dipakai oleh generate_id() milik Alexander untuk mengecek ID baru dan oleh pencarian judul milik Dhafi untuk mengambil resep setelah ID ditemukan di trie.

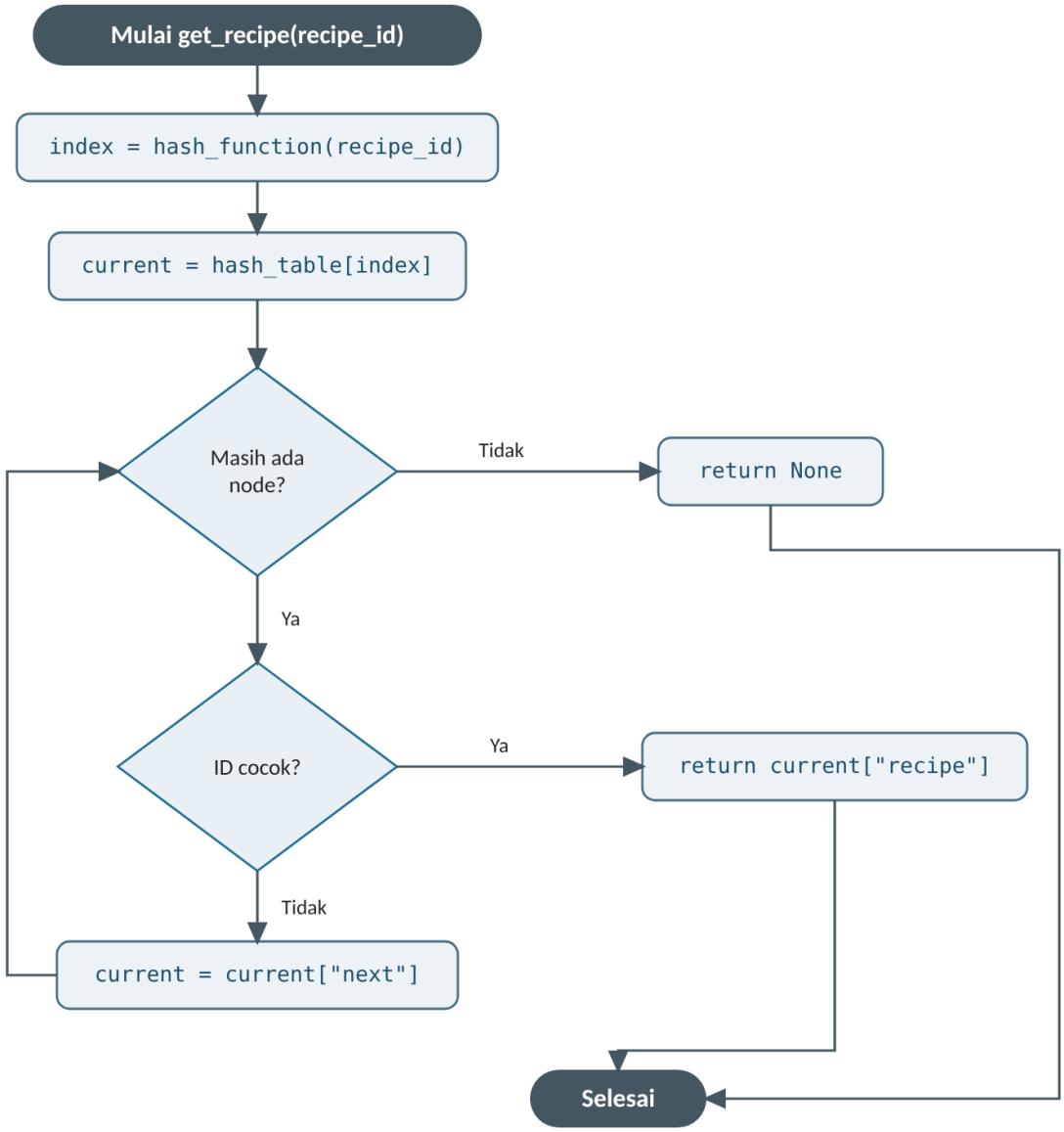
Masalah yang saya jawab adalah kecepatan pengambilan resep. Kalau setiap kali butuh resep program harus membaca cook_book dari awal sampai ketemu, kodenya memanjang dan pekerjaannya boros. Dengan hash table, posisi sebuah resep bisa dihitung langsung dari ID-nya lewat operasi modulo, lalu linked list menampung resep-resep yang kebetulan jatuh di slot yang sama tanpa saling menimpa.

Bagian	Rincian
Input	ID resep bertipe int, misalnya 561. Masukan ini datang dari dalam program, baik dari trie setelah judul ketemu maupun dari generate_id() yang sedang mengecek ID baru.
Process	hash_function() menghitung ID % 10 untuk menentukan slot 0 sampai 9, lalu get_recipe() berjalan menyusuri rantai simpul di slot itu selama masih ada simpul dan ID belum cocok.
Output	Dictionary resep yang dicari bila ketemu, atau None bila rantai habis sebelum ID ditemukan.

Tabel 1. Analisis IPO hash table dan linked list

2. Flowchart Fitur

Alur get_recipe() saya gamarkan pada Gambar 1. Setelah slot ditentukan, fungsi memeriksa simpul satu per satu: kalau simpulnya kosong berarti resep tidak ada dan hasilnya None, kalau ID di simpul itu cocok resep langsung dikembalikan, dan kalau tidak cocok fungsi pindah ke simpul berikutnya lewat kunci next lalu memeriksa ulang dari atas. Bentuk perulangannya while karena panjang rantai tidak diketahui dari awal; ada slot yang isinya satu simpul saja, ada yang dua.



Gambar 1. Flowchart fungsi get_recipe() pada data.py

3. Elemen Pemrograman Dasar

Variabel utama di bagian saya beserta tipenya saya daftar di Tabel 2. Yang menarik perhatian saya saat mengerjakan adalah current: variabel ini selalu bertipe dictionary, tetapi maknanya bergeser, kadang berdiri sebagai satu simpul berisi recipe dan next, kadang dipakai untuk berpindah. Konsep penunjuk yang biasanya dipelajari di materi lanjutan ternyata bisa dimodelkan murni dengan dictionary.

Variabel	Tipe	Isi / Contoh
hash_table	list	10 slot, diisi [None] * 10 sebelum dipakai
index	int	hasil hash_function(561), yaitu 1
current	dict / None	simpul {'recipe': {...}, 'next': {...}} atau None di ujung rantai
new_node	dict	simpul baru {'recipe': recipe, 'next': None} saat tabel dibangun
recipe_id	int	561

Tabel 2. Variabel pada hash table dan linked list

Operator yang paling sering bekerja di bagian saya adalah modulo, penentu slot dari setiap ID: 347 % 10 = 7, 821 % 10 = 1, 561 % 10 = 1. Dari dua contoh terakhir itu langsung terlihat kenapa linked list diperlukan, karena dua ID berbeda jatuh ke slot yang sama. Operator lainnya adalah perbandingan == untuk mencocokkan ID, dan pemeriksaan is None serta is not None untuk mendeteksi rantai yang habis. Keluaran bagian saya berupa dictionary resep yang dikonsumsi fitur lain, bukan sesuatu yang dicetak langsung ke layar.

Kutipan file data.py: operator modulo pada hash_function() dan list 10 slot

```
def hash_function(recipe_id):
    if recipe_id < 10:
        return recipe_id
    return recipe_id % 10

hash_table = [None] * 10
```

4. Percabangan dan Perulangan

Percabangan di make_hash_table() memutuskan satu hal penting: apakah slot masih kosong atau sudah diisi. Bila hash_table[index] bernilai None, simpul baru langsung ditempatkan sebagai penghuni pertama. Bila tidak, berarti sudah ada resep lain di slot itu, dan inilah yang disebut collision; simpul baru harus diantre ke ujung rantai. Percabangan if sederhana seperti ini juga muncul di hash_function() untuk ID di bawah 10 supaya tidak terpotong oleh modulo, dan di get_recipe() untuk mencocokkan ID di tiap simpul.

Perulangan while muncul dua kali dan keduanya menyusuri rantai. Versi pertama ada di make_hash_table(): while current["next"] is not None berjalan sampai simpul terakhir lalu menambatkan simpul baru di situ. Versi kedua ada di get_recipe(): while current is not None berjalan selama masih ada simpul untuk diperiksa. Perbedaan syaratnya disengaja, yang pertama mencari ujung rantai untuk menambah, yang kedua memeriksa seluruh rantai untuk mencari. Sementara itu for dipakai make_hash_table() untuk membaca 14 resep dari cook_book satu per satu saat membangun tabel, dan juga dipakai fitur pencarian metode milik Dhafi di bagian lain aplikasi.

5. Fungsi dan Struktur Data

Fungsi yang saya bahas adalah get_recipe(recipe_id). Parameternya satu, yaitu ID resep bertipe int, dan nilai baliknya dictionary resep bila ditemukan atau None bila tidak. Saya sengaja memakai None alih-alih pesan error, karena fungsi ini dipanggil dalam dua suasana berbeda: sebagai pencari biasa oleh fitur pencarian judul, dan sebagai pemeriksa keunikan ID oleh generate_id(). None menjadi bahasa bersama yang gampang diperiksa dengan is None oleh siapa pun yang memanggilnya.

Kutipan file data.py: fungsi get_recipe()

```
def get_recipe(recipe_id):
    index = hash_function(recipe_id)
    current = hash_table[index]

    while current is not None:
        if current["recipe"]["ID"] == recipe_id:
            return current["recipe"]

        current = current["next"]

    return None
```

Dari sisi struktur data, bagian saya contoh langsung pemakaian list dan dictionary berlapis. hash_table adalah list 10 slot, dan setiap slot menampung simpul berupa dictionary dengan dua kunci: recipe yang berisi dictionary resep utuh, dan next yang menunjuk simpul berikutnya atau None. Rangkaian next inilah linked list-nya, dibangun murni dari dictionary di dalam dictionary. Sementara cook_book milik Alexander menjadi sumber data yang saya baca saat membangun tabel, jadi list, dictionary, dan pola perulangannya benar-benar terpakai bersama di satu tempat kerja saya.

6. Penelusuran Kode dan Pengujian

Skenario yang saya telusuri adalah get_recipe(561), kasus nyata tabrakan di slot 1. ID 821 milik Fried egg dan 561 milik Chicken fried rice sama-sama menghasilkan angka 1 dari modulo, jadi keduanya berbagi slot dalam satu rantai. Penelusurannya di Tabel 3 menunjukkan resep yang dicari memang berada di simpul kedua, bukan yang pertama, sehingga perpindahan lewat next benar-benar ditempuh.

Langkah	Kode yang Dijalankan	Keadaan / Hasil
1	index = hash_function(561)	561 % 10 = 1, jadi index = 1
2	current = hash_table[1]	current menunjuk simpul ID 821 (Fried egg), penghuni pertama slot
3	if current["recipe"]["ID"] == 561	821 tidak sama dengan 561, kondisi gagal
4	current = current["next"]	current pindah ke simpul kedua, ID 561
5	if current["recipe"]["ID"] == 561	561 sama dengan 561, kondisi berhasil
6	return current["recipe"]	dictionary Chicken fried rice dikembalikan

Tabel 3. Penelusuran get_recipe(561) pada rantai slot 1

Pengujian saya bagi dua, langsung di Python untuk membuktikan perilaku fungsi dan lewat menu untuk membuktikan fungsinya berguna di program. Hasil pengujian langsung ada di Tabel 4; angka-angkanya diambil dari sesi uji yang benar-benar saya jalankan.

Pengujian langsung di Python	Hasil
hash_function(347)	7
hash_function(821)	1
hash_function(561)	1, tabrakan dengan 821 di slot yang sama
get_recipe(561)	{'ID': 561, 'title': 'Chicken fried rice', 'ingredients': ['rice', 'chicken', 'egg', 'soy sauce'], 'cooking_method': 'Fried'}
get_recipe(821)	{'ID': 821, 'title': 'Fried egg', 'ingredients': ['egg', 'oil'], 'cooking_method': 'Fried'}
get_recipe(999)	None, karena slot 9 tidak berisi ID 999
len(hash_table)	10

Tabel 4. Pengujian unit hash table

Baris get_recipe(999) penting saya sertakan karena membuktikan jalur gagal: slot 9 memang berisi resep lain bernomor 729, rantainya ditelusuri sampai habis, dan None dikembalikan tanpa program marah. Berikut tiga skenario lewat menu utama.

Skenario uji 1: mencari judul Chicken fried rice, ID 561 ditemukan di trie lalu diambil lewat get_recipe()

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 4

===== FIND RECIPE =====
1. Search by Cooking Method
```

```
2. Search by Title
3. Back
Choose an option: 2
Enter recipe title: Chicken fried rice

-------------------------
Recipe Number: 8
Title: Chicken fried rice
Ingredients: ['rice', 'chicken', 'egg', 'soy sauce']
Cooking method: Fried
-------------------------

===== FIND RECIPE =====
1. Search by Cooking Method
2. Search by Title
3. Back
Choose an option: 3

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

Skenario uji 2: mencari judul yang tidak ada, tetap berakhir rapi dengan pesan Recipe not found.

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 4

===== FIND RECIPE =====
1. Search by Cooking Method
2. Search by Title
3. Back
Choose an option: 2
Enter recipe title: Rendang
Recipe not found.

===== FIND RECIPE =====
1. Search by Cooking Method
2. Search by Title
3. Back
Choose an option: 3

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

Skenario uji 3: resep baru mendapat ID acak; keunikannya dicek oleh get_recipe() lewat generate_id() (dipadatkan)

```
===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 1

===== ADD RECIPE =====
Enter recipe name: Fried banana
Enter ingredients separated by comma: banana, flour, oil, sugar
Enter cooking method: Fried
Recipe added successfully.

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 5

===== ALL RECIPES =====

-------------------------
Recipe Number: 1
Title: Boiled egg
Ingredients: ['egg', 'water']
Cooking method: Boiled
-------------------------

-------------------------
Recipe Number: 2
Title: Fried egg
Ingredients: ['egg', 'oil']
Cooking method: Fried
-------------------------

     ... resep nomor 3 sampai 14 tampil dengan format yang sama ...

Recipe Number: 14
Title: Chicken soup
Ingredients: ['chicken', 'curry', 'coconut milk']
Cooking method: Simmered
-------------------------
-------------------------
Recipe Number: 15
Title: Fried banana
Ingredients: ['banana', ' flour', ' oil', ' sugar']
Cooking method: Fried
-------------------------

===== COOKBOOK SYSTEM =====
1. Add Recipe
2. Remove Recipe
3. Change Recipe
4. Find Recipe
5. Show All Recipes
6. Exit
Choose an option: 6
Goodbye.
```

7. Modifikasi dan Refleksi

Perbaikan yang saya lakukan ada di make_hash_table(), tepatnya cara menangani collision. Versi pertama saya menulis hash_table[index] = new_node begitu saja, dan saat data masih sedikit semuanya tampak baik-baik saja. Masalah baru muncul saat saya menguji dua ID yang bermuara di slot sama: resep yang datang belakangan menimpa penghuni lama, sehingga resep itu hilang dari hasil pencarian meskipun masih ada di cook_book. Perbaikannya adalah blok else yang berjalan menyusuri rantai dengan while sampai menemukan simpul yang next-nya masih None, lalu menyambungkan simpul baru di ujungnya. Sejak itu tidak ada resep yang hilang lagi.

Pelajaran terbesar saya dari kejadian itu: bug struktur data tidak selalu kelihatan saat datanya sedikit. Program saya ternyata lolos uji pertama semata-mata karena beruntung. Setelah itu saya membiasakan menguji dengan kasus tabrakan yang sengaja saya cari, seperti pasangan 821 dan 561 yang saya pakai di laporan ini, bukan menunggu kebetulan menemukannya.

Selama proyek saya memakai AI untuk bertanya cara kerja modulo dan pola penanganan tabrakan saat saya masih belajar konsepnya, lalu saya menulis kodenya sendiri dan menelusurinya manual dengan contoh angka seperti di Tabel 3. Semua hasil pengujian di atas saya jalankan sendiri dan tiap baris hash table ini bisa saya jelaskan tanpa melihat catatan.